CSCI2000U MIDTERM - SIDE A: EXAM TRAPS, UNIX, FILES, PERMISSIONS, GIT

EXAM TIPS (Fri Nov 6, 2:10pm, 1 hr)

MCQ + short + long answers. Tutorials (TE1-6) are "just like the exam". Concepts > syntax. Trace code line by line, write values down.

NOT tested: git log --pretty codes, Jupyter shortcuts, SSH key setup.

PROF'S A-OR-B TRAPS

`cp -r adir/ cdir` copies the CONTENTS into cdir (cdir/a.txt). `cp -r adir cdir` puts adir INSIDE cdir (cdir/adir/a.txt). (Slides = macOS rule.)

`print(a,b,sep="_",end="!")`: sep goes BETWEEN, end goes AFTER the whole line (no newline).

`"aaaa".count("aa")` = 2 and `"banana".count("ana")` = 1: matches never overlap.

No `return` gives `None`. `lst.sort()` gives `None` (use `sorted(lst)` for a new list).

Slices are VIEWS. `copy()`/`flatten()` are copies. `ravel()`/`reshape()` are views.

`.gitignore` does NOT untrack a committed file. `git restore f` DESTROYS edits. `git revert` ADDS a commit.

`axis=0`: one answer per COLUMN. `axis=1`: one answer per ROW (slide wording is misleading).

`open(f,"w")` ERASES the file even if you write nothing. Use `"a"` to append.

UNIX BASICS (L01-L03)

UNIX: Bell Labs (Thompson, **Ritchie**), rewritten in **C**. **Linux = Unix-like** (Torvalds), not UNIX itself. macOS is UNIX. Windows: use WSL.

Terminal = the window. **Shell** = program that runs your commands (bash, zsh; `echo $0`). **Prompt** = text before cursor (`$`). **CLI** = text interface.

Command = `command -options arguments`: `ls -l docs/`. Options combine: `ls -la`. Case-sensitive.

`/home` users, `/etc` config, `/bin` programs, `/tmp` temp. One tree rooted at `/`.

PATHS (L02)

Absolute starts with `/` (or `~`). **Relative** does not. `~` = home, `.` = here, `..` = parent, `cd -` = previous dir. `..` at `/` stays at `/`.

Trace: start `/home/sam/docs`; `cd ../data` ends `/home/sam/data`; `cd ../..` ends `/home`. Count each `..` as one step up.

Hidden files start with `.` (show with `ls -a`).

SHELL COMMANDS (L03, TE2)

`pwd`	print working directory
`cd dir` `cd ..` `cd ~` `cd -`	change dir / up / home / previous
`ls`	list. `-l` long, `-a` all (hidden), `-S` by size, `-r` reverse, `-R` recursive, `-F` mark type, `-h` human sizes
`ls -RSral`	recursive + size-sorted + reversed + all + long
`ls main*`	wildcards: `*` any chars, `?` one char
`mkdir d`	make dir. `-p a/b/c` makes parents, no error if exists
`mkdir project{1..3}`	brace expansion: project1 project2 project3
`touch f`	create empty file (or update time)
`echo "x" > f`	**overwrite** f. `>> f` **appends**
`cat f` `head f` `tail f`	show file / first / last lines
`man cmd`	manual page. `which cmd` where it lives

FILES (L04, TE2)

`cp a b`	copy file. `cp -r dir new` copies a directory (needs `-r`)
`mv a b`	move OR rename. If b is an existing dir, a goes inside it
`rm f`	delete file (no trash, no undo). `rm -r dir` deletes dir + contents

PERMISSIONS (L03, TE2)

`ls -l`: `-rwxr-x--x`. 1st char = type (`-` file, `d` dir). Then owner / group / others, 3 each (r w x).

r=4 w=2 x=1. 7=rwx 6=rw- 5=r-x 4=r-- 2=-w- 1=--x 0=---. **755**=rwxr-xr-x **644**=rw-r--r-- **640**=rw-r----- **600**=rw------- **712**=rwx--x-w-.

Symbolic: `u g o a` + `+ - =` + `r w x`. `chmod u+x f`, `chmod g+r *.txt`, `chmod a=r f`, `chmod u=rw,go= f`. Numeric: `chmod 640 f`.

Adir needs `x` to `cd` into it.

VERSION CONTROL (L04-L05)

VCS vs backup: history of changes, who/why, restore any version, collaborate.

Git is distributed: every clone has the full history. Commits are **snapshots** of the whole project.

3 areas: working directory -> staging area (index) -> repository (`.git`). Flow: edit, `add`, `commit`.

File states: untracked, modified, staged, unmodified (committed).

`git init` = turn a folder into a repo. `git clone url` = copy a remote repo. Git = tool, GitHub = host. Give GitHub the **public** key, never the private.

GIT COMMANDS (L05-L07, TE3, TE4)

`git config user.name "N"`	`--global` all repos, `--local` this one. `git config --list`
`git init`	new repo. `git clone <url>` copy remote
`git status`	what is untracked / modified / staged
`git add f`	stage f. `git add .` or `--all` stage everything
`git commit -m "msg"`	save staged snapshot. Needs something staged
`git log`	history, newest first. `--oneline`, `-n 3`, `--name-only`, `-1`
`git show HEAD`	one commit + its change. `HEAD~1` parent, `HEAD~2` two back
`git diff`	working copy vs **staged**. Empty after `add`
`git diff --staged`	staged vs **last commit** (what the next commit holds)
`git rm f`	delete from disk + stage the deletion
`git rm --cached f`	stop tracking, KEEP file on disk

UNDO (L06-L07, TE3, TE4)

`git restore --staged f`	**unstage**. Edits stay in the file
`git restore f`	**discard** working edits (file = last commit). Permanent
`git checkout HEAD f`	old way: restore f from HEAD. `HEAD~2 f` = 2 commits ago
`git revert --no-edit HEAD`	NEW commit that undoes HEAD. History grows (3 -> 4)

STAGED + MODIFIED (TE3 #4)

```
echo -n "a" > n.txt; git add n.txt
  # staged: a
echo -n "b" >> n.txt; git status
  # staged (a) AND modified (ab)
git add n.txt; git commit -m "ab"
  # now unmodified
```

`git add` stages the file AS IT IS at that moment. Edits made after `add` are NOT in the next commit until you add again.

.GITIGNORE & MISTAKES

`.gitignore` lists patterns (`*.log`, `data/`, `secret.env`). Affects **untracked** files only.

Committed secret: `git rm --cached secret.env`, add it to `.gitignore`, `git add .gitignore`, `git commit`.

Order of safety: `restore --staged` (safe) < `rm --cached` (safe) < `restore f` / `rm f` (lose edits).

CSCI2000U MIDTERM - SIDE B: PYTHON, STRINGS, FUNCTIONS, DATA STRUCTURES, NUMPY, FILES

PYTHON BASICS (L08, TE4)

Expression produces a value (`3+4`). **Statement** does something (`x=5`, `if`, `print()`). Jupyter shows only the LAST expression of a cell; use `print()` for the rest.

Types: `int` `float` `str` `bool`. `type(x)`. Convert: `int("5")`, `float("2.5")`, `str(5)`.

Operators: `/` float, `//` floor, `%` remainder, `**` power. `7/2`=3.5 `7//2`=3 `7%2`=1 `2**3`=8. `=` assigns, `==` compares.

`and` `or` `not`; `==` `!=` `<` `<=`. `(x>10) and (y<=3)`. Comments: `#`.

`import math`: `math.sqrt(144)`=12.0, `math.isqrt(144)`=12, `round(3.14159,3)`=3.142.

Dynamic typing: type belongs to the value, `x=5` then `x="hi"` is fine. **Strong typing:** no silent mixing, `"5"+3` is a `TypeError`.

STRINGS (L10, TE5)

Index from 0; `s[-1]` last. Slice `s[a:b]` includes a, EXCLUDES b. `s="data_science"`: `s[5:]`="science", `s[:4]`="data", `s[::-1]` reversed, `s[::2]` every 2nd.

Immutable: `s[0]="J"` is a TypeError. Methods return NEW strings: `s.upper()` leaves `s` unchanged.

`+` joins, `*` repeats, `len(s)`, `"th" in s`, `s.lower()` `.strip()` `.replace(a,b)` `.split(",")` `",".join(lst)` `.find(x)` `.count(x)` `.startswith(x)`.

f-string: `f"{name} scored {x:.2f}"`. Escapes: `\n` newline, `\t` tab.

CONTROL FLOW (L09, TE5)

`if / elif / else`: stops at the FIRST true branch. Separate `if`s ALL run. Ternary: `"pos" if x>0 else "neg"`.

`range(n)` is 0..n-1. `range(1,10,3)` = 1 4 7. `for x in lst:`. `for i,v in enumerate(lst)` gives (index, value). `while cond:` must change cond. `break`, `continue`.

Comprehension: `[x*x for x in range(7)]`, `[v for v in vals if v>7]`.

```
total = 0
for i in range(1, 6): total += i  # 15
i = 1; out = []
while len(out) < 5:
    out.append(3*i); i += 1
# out = [3, 6, 9, 12, 15]
def is_leap(y):
    return (y%4==0 and
            (y%100!=0 or y%400==0))
```

TUTORIAL OUTPUTS (check yourself)

`print(7/2, 7//2)` = 3.5 3. `[v for v in [3,6,9,12] if v>7]` = `[9, 12]`.

`{1,2,3}|{3,4}` = `{1,2,3,4}`; `&` = `{3}`. Average of 90 and 85 = `87.5`.

`[n**2 for n in range(0,11,2)]` = `[0, 4, 16, 36, 64, 100]`. `power(2,5)` = 32.

Standardize `[[1,2,3],[4,5,6]]` by column = `[[-1 -1 -1],[1 1 1]]`.

FUNCTIONS (L09, TE5)

`def f(a, b=1):` default parameter. Call by keyword `f(b=2, a=3)`. **No return = returns** `None`.

`*args` collects extra positional args into a **tuple**. `**kwargs` collects keyword args into a **dict**. `f()` with `*args` gives `()`.

`lambda x: 2*x` = small unnamed function. Higher-order: `apply_twice(f,x)` is `f(f(x))`; `apply_twice(lambda t:2*t, 3)` = 12.

Recursion: base case + a call on a smaller input. No base case = RecursionError.

```
def factorial(n):  # (5) -> 120
    if n == 0: return 1
    return n*factorial(n-1)
def add_all(*args):  # (1,2,3,4) -> 10
    return sum(args)
def greet(name, msg="Hi"):
    return f"{msg}, {name}!"
w = ["bb", "a", "ccc"]
sorted(w, key=lambda s: -len(s))
  # ['ccc', 'bb', 'a']
```

PRINT & ERRORS (L09)

`print(*objs, sep=" ", end="\n")`. Real default `sep` is ONE SPACE (slide shows `''`). `print("A","B",sep="_",end="!")` prints `A_B!` with no newline.

Traceback: read from the BOTTOM: error type + message, then the line. `NameError` `TypeError` `IndexError` `KeyError` `ValueError` `ZeroDivisionError` `SyntaxError`.

LISTS, TUPLES (L11, TE6)

list `[]` ordered, mutable. `append(x)`, `insert(i,x)`, `pop()` removes+returns LAST, `pop(i)`, `remove(x)`, `reverse()`, `len`, `in`, `copy()`. `l[1]=20`, `l[-2:]=[30,40]`.

`sort()` in place returns `None`; `sorted(l)` returns new list. `l*3` repeats the list.

tuple `()` immutable: `t+(4,)` makes a NEW tuple. `(1,)` one-element.

Aliasing: `b=a` same object. `a.copy()` is SHALLOW: inner lists still shared. `a=[1,[2,3]]; b=a; c=a.copy(); a[1][0]=99` -> a, b AND c all show 99.

Class challenge: `fruits.insert(0, fruits.pop())` moves the last item to the front.

SETS, DICTS (L11, TE6)

set `{}` unique, unordered, no index. `set([1,2,2,3])` = {1,2,3}. `a|b` union, `a&b` intersection, `a-b` difference, `add`. `{}` is an empty DICT; empty set = `set()`.

dict `{k:v}`: `d[k]`, `d[k]=v` (add OR overwrite), `del d[k]`, `k in d`, `d.keys()` `.values()` `.items()` (tuples). Repeated key keeps the LAST value.

Nested: `recs=[{"name":"Ada","score":90},...]`. `[r["name"] for r in recs]`, `sum(r["score"] for r in recs)/len(recs)`, `{r["name"]:r["score"] for r in recs}`.

Unique sorted tags: `sorted({t for r in recs2 for t in r["tags"]})`.

NUMPY (L12-L13, TE6)

`import numpy as np`. Array = **homogeneous**, **contiguous** memory. Vectorized ops (no loops) are faster in practice, NOT lower Big-O (still O(n)).

`np.array([1,2,3,4])`. `.shape` (tuple, `(4,)`), `.ndim` (int), `.dtype`, `.size`. `np.zeros((2,3))` shape is a TUPLE; also `ones`, `full`.

`np.arange(0,6)` ints like `range`. `np.linspace(0,1,5)` evenly spaced floats, includes end. `np.arange(6).reshape(2,3)`; `reshape(-1,1)` = column (-1 infers).

`arr*3` is elementwise; `list*3` repeats. `arr+list` works.

Index `a[i,j]`; row `a[0,:]`; column `a[:,0]`; `a[1:3,::2]`. **Mask:** `x[x>0]` keeps True positions.

ufuncs (elementwise, return new array): `sqrt` `abs` `add` `multiply` `sin` `floor` `ceil`.

`sum` `mean` `min` `max` `std`. No axis = ONE number. `axis=0` collapses rows (**one per column**). `axis=1` one per row. `U=arange(6).reshape(2,3)`: `U.mean(axis=0)`=[1.5 2.5 3.5].

Broadcasting: compare shapes from the RIGHT; OK if equal or one is 1 (the 1 stretches). (3,1)+(4,)=(3,4). (2,3)+(3,)=(2,3). (2,3)+(2,) = ERROR.

`np.vstack([R,R])` (3,2)->(6,2). `np.hstack([R,c])` needs same rows (R (3,2) + c (3,1) -> (3,3)). Standardize: `(M-M.mean(axis=0))/M.std(axis=0)`.

VIEWS vs COPIES (L13)

VIEW (shares memory, changing it changes original)	`a.view()`, slices `a[:2]` `a[0,:]`, `reshape()`, `ravel()`
COPY (independent)	`a.copy()`, `flatten()`
`reshape()` vs `resize()`	reshape returns a new view, `a` unchanged. `resize` changes `a` in place

`b=a[1:3]; b[0]=99` changes `a`. `c=a.copy(); c[0]=-1` does not. A view is just a pointer to the same data: both "point to the same place in memory".

FILES & DATA SCIENCE (L14)

`with open("f.csv","w",newline="") as f:` **closes the file automatically** at the end of the block. Modes: `r` read, `w` write (**erases** old data), `a` append.

`import csv`; `csv.writer(f).writerow([1,"Ada",5000])`; `for id,name,val in csv.reader(f):` rows are lists of **strings**. `os.getcwd()` = where the file goes.

CSV = comma-separated plain text, opens in any editor. `pd.read_csv("f.csv")` loads a **DataFrame** (main Pandas structure).

Process: real world -> collect -> **wrangle/clean** (munging, >80% of time) -> **EDA** (graphs, summary stats, outliers, missing data; NO hypothesis, iterative) -> analysis/model (explanatory = causes) -> communicate -> product (feedback loop).